A What's an Image?

What's an image? It is a representation that encodes how much light exists at each place on a regular grid.

For better or for worse, images will come in all sorts of forms throughout this course and the code you will use will assume different formats or conventions. This will be a source of **great annoyance** throughout the course. I've been doing computer vision for over a decade and it's still an annoyance.

A.1 Shapes of Images

Images come in a variety of shapes. Depending on who's in charge, either the rows come first or the columns come first in terms of how it's laid out in memory, referenced in terms of array access, and in terms of how "size" is denoted. Once you add in colors, the number of options gets even worse! We'll usually refer to a grayscale image as a $h \times w$ matrix/array and a color (RGB) image as a $h \times w \times 3$ matrix/array.

Here are three fairly common issues with image shapes.

- In a lot of material, sizes will be *implied*: a $H \times W$ color image is actually a $H \times W \times 3$ array.
- The **order** of colors in a color image isn't always standardized. Most libraries assume that the colors are stored in red, green, and blue (i.e., `I[:,:,0]` is red) but for historical reasons, opencv stores things in blue, green, red order.
- Some images have a fourth channel, called *alpha*. This represents opacity. An alpha of 0 corresponds to transparent, and an alpha of 1 (or maximum value) corresponds to opaque. Some image formats store this and some libraries return it. If you don't want it, you can usually just ignore the channel.

A.2 Values of Images

The meaning of data depends on context and convention. The four bytes `0xF30F58C1` mean:

- $-1.13570952431 \times 10^{31}$ if you interpret it as a floating point number
- 4077869249 if you interpret it as an unsigned integer
- -217098047 if you interpret it as a signed integer (using two's complement)
- `addss xmm0,xmm1` if you interpret it as an instruction to an Intel x86-64 processor.
- ó<*Shift-in control code*>XA if you treat it as an a string and decode it with the Latin-1 encoding

But really, what does it actually mean? It fundamentally depends on who wrote it and why.

Images are no different and this is a source of lots of bugs. Sometimes your bugs will be that two pieces of code are completely correct, but they expect different formats. Let's look at a few common values/representations each pixel v can take and what you need to look out for when working with each:

- **An unsigned 8-bit integer** $\in \{0, 1, \ldots, 255\}$ where 0 and 255 are the minimum and maximum amounts of light. This is how things are commonly stored for images – humans can't spot small differences in light and so there's no value in storing past 8-bits of precision (although of course there do exist 16-bit integer images, for instance in medicine and astrophysics).

Danger: Beware that math is integer math! Integers are annoying since they lose precision (e.g., $2/3 \rightarrow 0$) and overflow ($9 \times 40 \rightarrow 104$). Doing all the math in uint8 can be *much* faster than using floating point numbers; however, it's easy to write code that doesn't work. When you start out, convert *everything* to floating point (preferably double precision/`np.float64`) where there are fewer gotchas. Numpy helps a lot by trying to nudge you to do floats, but depending on what precise operations you do, you can find yourself still in integer land.

Note: It may be helpful to think of this as just a representation where you're storing things that range from 0 to 1, and to get that value you need to divide by 255. It's often common for instance for people to store probabilities as integers from [0,255]. To get the correct value, you just divide by 255 (after converting to a float).

- **A floating point number** $\in [0, 255]$ where 0 and 255 are the minimum and maximum amounts of light. This is quite common when you want to do stuff like take the average of things without worrying about overflow. You can immediately convert back to the nearest uint8 integer and things will be fine.

  Danger: Beware that the maximum light is 255, not 1! This leads to two common issues: (1) some systems may think that the maximum value of the image is 1 and will clip any value bigger than 1 to be equal to 1. So you may have a pixel that has a value 42, and it'll be forced to 1. This results in an image that's almost all white. (2) some equations/formulas assume values fall between 0 and 1 and will screw up outside this range – e.g., squaring a positive number x reduces its value if $x \in [0, 1]$ but increases it if $x > 1$.

- **A floating point number** $\in [0, 1]$ where 0 and 1 are the minimum and maximum amounts of light. This is the most common way floating point images are represented since 255 is a totally arbitrary convention – there's no reason why you need 8 bits of precision – and a lot of things are better expressed in some sort of common scale. $[0, 1]$ is about as inoffensive as it gets.

  Danger: Beware that the maximum light is 1, not 255! If you convert this directly from a float to an unsigned integer, you'll end up with pixels that are 0 and 1 (which are the darkest and second-darkest for `uint8`). You'll end up with an image that's completely black!

- **An unsigned integer** $[0, 1, \ldots, K]$. This is less common, but can indicate:

  1. regions in an image (e.g., from a system that divides the image into a set of regions corresponding to objects) where all pixels that have the same value are part of the same region.
  2. categories in an image (e.g., from an object detector) where $v = 2$ might indicate that there's a horse at that pixel or $v = 10$ might indicate that there's a dog at that pixel.
  3. the index into a palette (like Paint by numbers) where $v = i$ might indicate that the pixel is colored with color i (e.g., $0 =$ blue, $1 =$ red).

  Danger: Beware that this normally isn't a real image! You can look at it, but think of it like a map that's been colored. Tanzania being colored purple on a globe doesn't mean that Tanzania is purple.

- **A floating point number** $\in [-\infty, \infty]$ that represents some physical quantity. This could be the distance from the camera plane, the velocity at a location, or anything else.

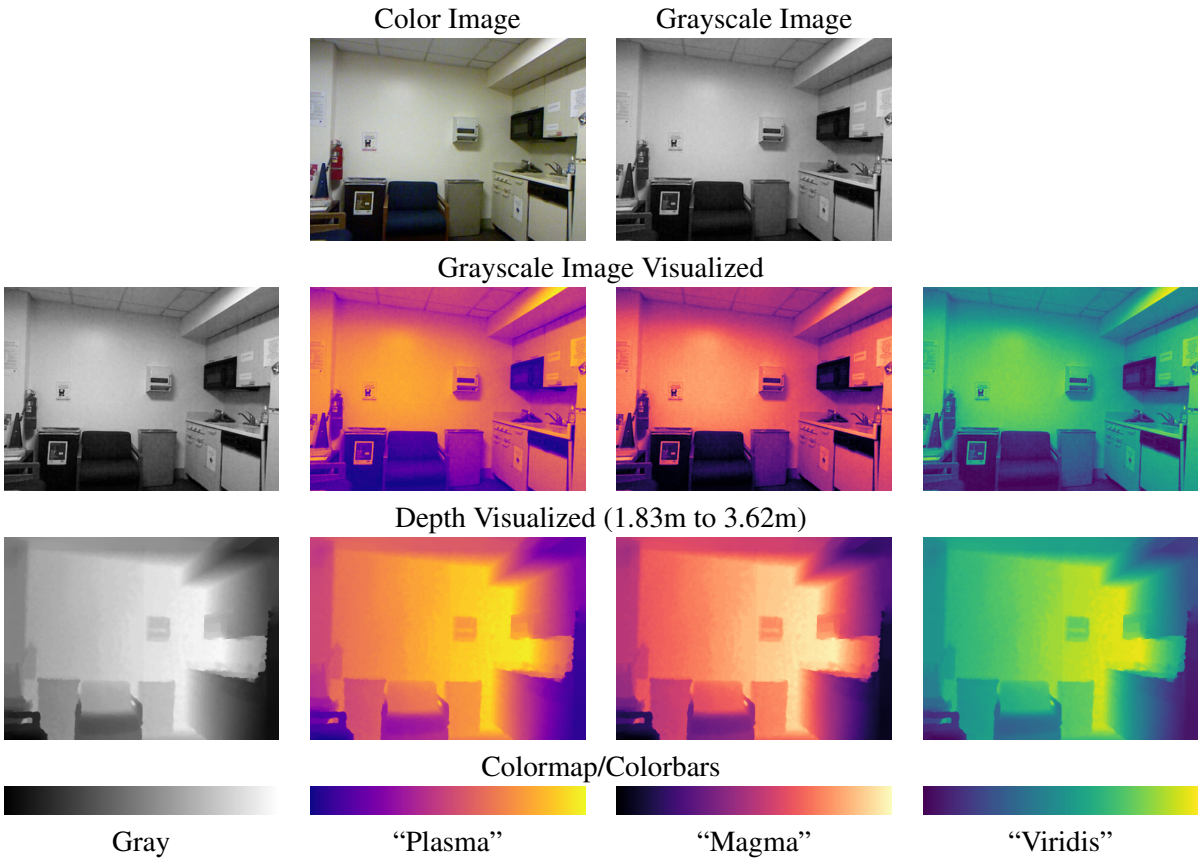

Figure 7: Colormapping a grayscale image and its corresponding depth (distance to the camera along the viewing axis) as measured by a ranging sensor. Both are shown with four colormaps that matplotlib provides (viridis is the default option).

> *Danger: keep track of units.* You probably won't encounter this in the course as an input. But you will encounter this as an intermediate step of a calculation. It's important to keep track of what the image means. Otherwise you have something that looks interesting but doesn't have meaning.

A.3 False Color/Colormapped Images

Many of the images you'll deal with in the course are *false color* images. These are ways of showing some data that is like an image but isn't an image for which there's a natural way of showing things (e.g., the amount of brightness). You can think of it as a way of converting some arbitrary image with arbitrary type into a standard RGB image.

You've seen many of these without thinking about it. When people visualize population density in the US, you can't actually "see" 40 people per square kilometer. These are converted into visualizations where different values are colored differently: maybe areas with few people are drawn as white, and areas with many people are drawn as red. The same idea applies to looking at "images" from Magnetic Resonance Imaging (MRI) machines, depth from a camera, or magnetic fields on the Sun. You may encounter this while trying to visually debug a program since many variables inside the code you write are actually images.

The two ingredients to a false color image are: a piece of imaging data to visualize, and a colormap. The image can be anything so long as there is a single value. The colormap is a mapping from every number

between 0 and 1 to a color. In Figure 7, for instance the plasma map slowly changes from purple to yellow. The number 0 corresponds to purple; the number 1 corresponds to yellow, and 0.68 is some shade of orange.

Images don't naturally range from 0 to 1 – the depths of objects in Figure 7 range from 1.83 to 3.62m – so there is one more step needed – rescaling the image values. Let's assume the image data is all real-valued and ranges from $v_{\min}$ to $v_{\max}$. When the image is colormapped, every single pixel v in the original image is replaced by the corresponding color for $(v - v_{\min})/(v_{\max} - v_{\min})$. The halfway point in the original data is $(v_{\max} - v_{\min}) * 0.5 + v_{\min}$ (i.e., half the range plus the minimum) and if you stare at this, this value will correspond to 0.5, and so therefore to the color halfway through[3]

While $v_{\min}$ and $v_{\max}$ are often automatically set via the data, it is sometimes important to manually set them. This is crucial if you are comparing two colormapped images. If the values of $v_{\min}$ and $v_{\max}$ are different between images, then colors mean totally different things! Imagine if we colormapped weather maps from July and from January. If we decide on what "yellow" and "blue" mean independently, one map's yellow might correspond to a lower value than another's blue.

[3] A final technical nuance is that colormaps are instead frequently stored as tables. While many colormaps have an analytical form, this is not always the case. Thus, it is common practice to instead store a "look-up" table which contains colors 0 to $N - 1$. One therefore does a final remapping from 0 to 1 to 0 to $N - 1$.

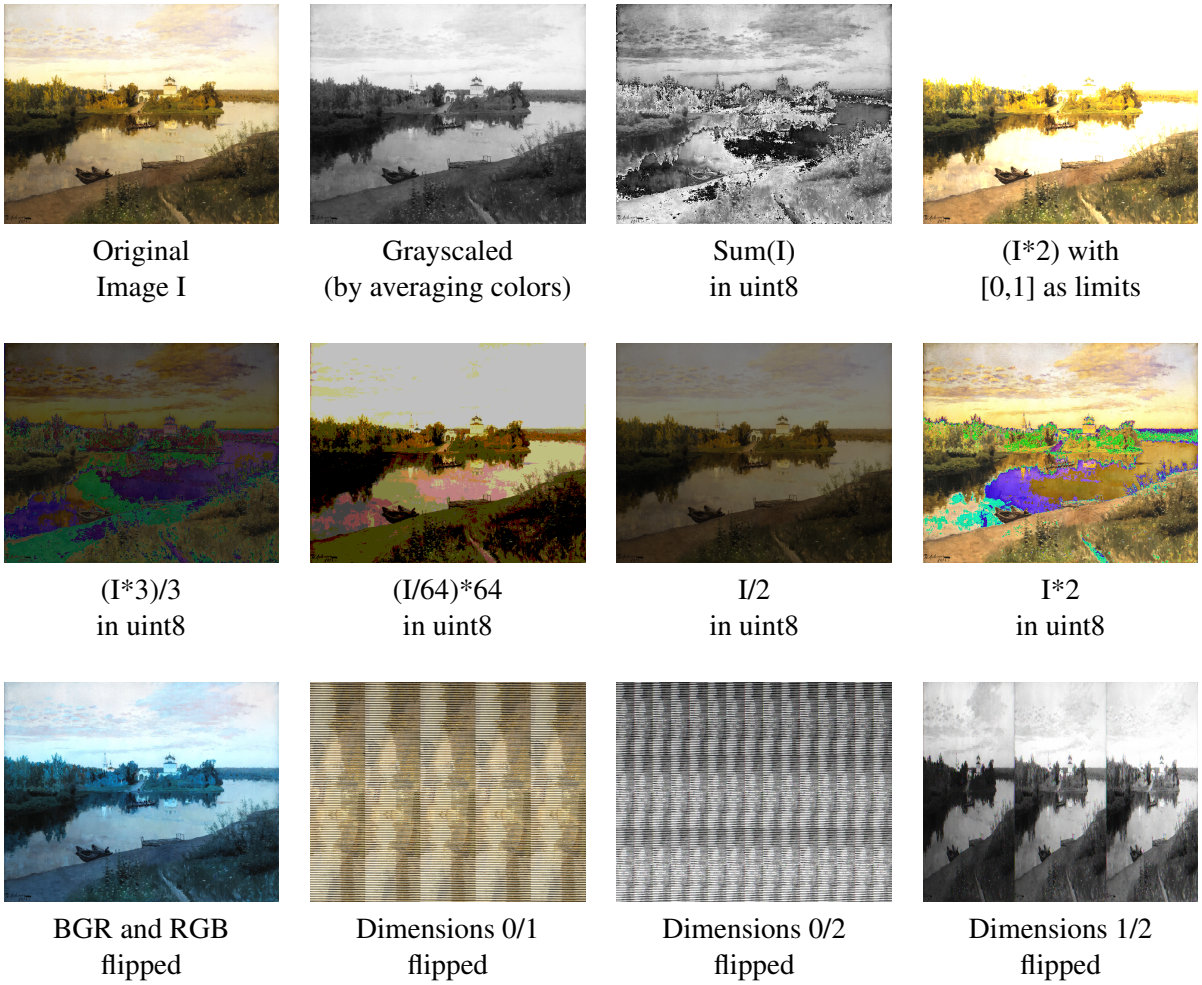

Figure 8: **Visually identifying bugs.** Particular types of bugs have particular visual appearances. Here are a few common issues. Some are due to doing operations on uint8 that don't act like normal numbers (indicated by a colors being off). Some are do to indexing the image incorrectly (e.g., treating rows and columns incorrectly).

B Debugging Hints

B.1 Visually Identifying Bugs

Many bugs can be identified quickly entirely visually in the end product. Fortunately, we have a lot of experience writing code with bugs. We've made a montage of common image bugs in Fig 8. It's rare for you to have exactly one of these bugs. But the type of behavior suggests different bugs.

1. **I can't see anything:** If you expect an image with something, but it's just black or just white, this is often a type issue, a NaN somewhere, an image that has no range (i.e., everything is indeed the same value), or an image with too big of a range (i.e., most values are $\sim 10^1$ and one value is $\sim 10^{20}$).
2. **Colors gone wrong:** If general shape looks right but the colors are wrong, it's likely that your math is doing something that doesn't quite work out correctly. Common causes include things like:
   (a) UINT8 math: Remember that integers on computers are modular. For instance UINT8 192 (light

gray) + 128 (medium gray) = 64 (dark gray).

This has issues especially in colors – [192,64,64] plus [32,128,192] = [224,192,0] or red plus blue equals not purple! You might do something like take the average of a blue and red (i.e., ([192,64,64]+[32,128,192])/2) and end up with a dark yellow like this instead of something vaguely magenta-y like this.

You prevent this by having enough precision to do the math you need. I recommend using a float (preferably double-precision) for all your math for when you start out. You can also use 16-bit uints. But the speedup compared to doubles (under an order of magnitude) is not worth the increased chance of edge cases for when you start out. For the curious: when your computer does these averages (e.g., to blend two images) it may be doing it via a CPU instruction (pavgb) that temporarily uses 9 bits!

(b) Some formula goes from 0 to 2 **instead of** from 0 to 1. If you multiply the image by 2, anything that is really intensity 127.5/0.5 will really be 255/1!

(c) A formula that's missing an important constant. For instance, if you compute an average by summing but forget to divide by a constant, you'll end up with something funny.

3. **Right type of color, but shapes gone wrong or points in the wrong position:** If the colors on average look OK, but the shape looks wrong (e.g., zig-zag patterns like an old TV, the image duplicated but squished), the shape of your image is being interpreted incorrectly. You can have this same issue if you're trying to point to locations on the image (e.g., where eyes, locations of interest, or corners of a box are).

If you don't have an image that you can pass in, generate fake data where you can identify what the answer should be. A common trick is to use `meshgrid` to generate images where the columns ascend and rows descend. Before you throw it in, identify – what things do you expect?

B.2 Visual Debugging

You'll probably write some buggy code! Don't panic. Here are a few things you should do:

1. **Design defensively beforehand.** Write small functions that do exactly one thing. Save results along the way in a format that is easy to understand. Load your results to make sure they load correctly.

2. **Assume every line of code could have a bug.** The bug is almost always something boring. In fact, because you're likely to look at the places for an *interesting* bug, any long-lasting annoying bug will probably not be there. It's often useful to write a separate script from scratch that loads in the data. Writing a second time is faster a little slower but prevents re-introduction of typos that are correct code but which are wrong.

3. **Use a debugger.** If you use an IDE, use breakpoints; if you don't, try pdb. If you print debug, you can't ask follow-up questions without re-running the program. This increases the amount of time you spend waiting for that code to finish.

4. **Check what's in the data!**

   OK, so you've got the program stopped and you have a variable named `X` that you think is suspect. What should you do?

(a) Check the data type (`X.dtype`)! You might have set the data type to one type, but you might have gotten the data from somewhere else. A lot of stuff will just screw up due to this. If it's not the type you expect, be skeptical and try to find out why.

(b) Check for not a numbers (NaNs) (`np.sum(np.isnan(X))` or `np.isnan(X).sum()`). NaNs emerge out of things like division by zero in floating point arithmetic, square roots of negative numbers. They get *everywhere* because NaN $\circ$ x = NaN for any x and (basically) operator $\circ$.

(c) Check the range: `X.min()` or `np.nanmin(X)` and `X.max()` or `np.nanmax(X)`. Do they make sense?

(d) Check the average value: `X.mean()`. Does this makes sense?

(e) Check the fraction of items above/below a threshold. If the value should almost always be positive, try `np.mean(X>0)`.

5. **Plot!** Your visual cortex is large and quite well-developed. You should visualize things even if we don't ask you to. This maximizes the chance you catch the bug earlier rather than later. Here are a few options:

   (a) Matplotlib imsave: `plt.imsave(filename,matrix)`
   This makes a false color image and saves it. This automatically scales things so that one end of the color scheme is the minimum and the other end is the maximum. Set `vmin` and `vmax` if you want different behavior. You'll implement a version of this later in the homework. You can select a colormap from the documentation on colormaps. The default colormap in matplotlib is called *viridis* and is pretty good.

   (b) Matplotlib imshow: `plt.imshow(X)`
   This shows the result as a figure.

   (c) Rescaling things to [0,255] to make an image that you can save.
   `((X-X.min()) * 255 / (X.max()-X.min())).astype(np.uint8)`
   This is grayscale so considerably less fun but also works.

   (d) Re-normalizing. If you can't see anything, play with the contrast. Do `X**0.5` to make larger values smaller (for $X > 1$).

6. **Develop an intuition for things to be skeptical of.** This takes a while. I'll give a few:

   (a) Values are rare at the min/max value for natural signals. It is rare for a normal image to have cyan in the image. It's rare for lots of the values to be close to 1.0 for floats or 255 for uint8.

   (b) NaNs are danger. Treat an *unexplained* NaN as a smoke detector beeping in your code.

   (c) Contrastless visualization. Common causes: You have a NaN; your range is 0; you have a single large value in one pixel (e.g., all pixels are in the range 0-3; one pixel is 9001).

   (d) Stuff that doesn't line up. If you have two pieces of data and they *should* line up but don't, there's probably an indexing thing that will get you later on.